This test uses html_response/2 to prove that a 200 status is returned along with an HTML body. The HTML is returned as a string, so we use regular expressions to test against it.

Each line like assert html =~ "a string" proves that the HTML code is inside of the response body. It's a good practice to test only the key elements on your page. If you test too many elements, then your test can become brittle. Brittle tests are painful over time because updates can break unrelated tests.

Test Post Requests

Phoenix makes it just as easy to test POST requests as it is to test GET requests. However, our test will be slightly more complex due to how our endpoint works.

Our endpoint makes an outbound HTTP request to Twilio, so we need to use Bypass to set up our test. Add the following describe block to your test module:

phone_app/test/phone_app_web/controllers/message_controller_test.exs

```
Line 1  alias PhoneApp.Conversations.Schema.SmsMessage
        -
        -  describe "POST /messages/new" do
        -    test "invalid params is rejected", %{conn: conn} do
      5  -      conn = post(conn, ~p"/messages/new", %{})
        -      assert html_response(conn, 200) =~
        -               Plug.HTML.html_escape("can't be blank")
        -    end
        -
     10 -    test "valid params creates a message", %{conn: conn} do
        -      bypass = Bypass.open()
        -      Process.put(:twilio_base_url, "http://localhost:#{bypass.port}")
        -
        -      Bypass.expect_once(
     15 -        bypass,
        -        "POST",
        -        "/Accounts/mock-account/Messages.json",
        -        fn conn ->
        -          conn
     20 -          |> put_resp_header("Content-Type", "application/json")
        -          |> resp(201, File.read!("test/support/fixtures/success.json"))
        -        end
        -      )
        -
     25 -      params = %{message: %{to: "+1111-222-3333", body: "Test"}}
        -      conn = post(conn, ~p"/messages/new", params)
        -      assert redirected_to(conn, 302) == "/messages"
        -      assert PhoneApp.Repo.aggregate(SmsMessage, :count) == 1
        -    end
     30 end
```